

AGRIVO — Guide technique de l'app mobile (pour Christ)

Spécification de référence — version du 6 juillet 2026, mise à jour le 7 juillet (backend v1.3.0 EN PROD), rédigée par Anael (chef de projet). Objectif : permettre à un dev mobile expérimenté de **repandre l'app mobile existante et l'aligner** sur la vision AGRIVO sans avoir à poser de questions. L'app est déjà commencée : on ne repart pas de zéro, on réadapte (voir §7). Backend de référence : <https://agrivo-io.vercel.app> (v1.3.0 — les nouveautés backend qui te concernent sont dans l'encadré §4.6). Jury : samedi 11 juillet 2026 — priorité absolue aux écrans du golden path.

⚠ MISE À JOUR STRATÉGIQUE IMPORTANTE (7 juillet, v1.3.0) — à lire avant tout. La PWA web fait désormais la capture GPS RÉELLE au bord du champ (mode « Tour de champ GPS réel » : `navigator.geolocation.watchPosition`, waypoints en marchant, fermeture de polygone, RFC 7946). Autrement dit : **une seule application (la PWA), installable depuis le navigateur, du bureau au champ, sans passer par un store** — ce que faisait ta valeur unique mobile est maintenant couvert par la PWA sur un téléphone.

Ce que ça change pour toi :

- **Décision à trancher avec Anael AVANT de coder** (réponds aux 10 questions du §7.1) : si ton app native existante fait DÉJÀ tourner caméra + GPS + carte, on la garde (le GPS natif en arrière-plan et l'anti-mock sont un vrai plus). **Sinon, on démontre le terrain sur la PWA** (installée sur un téléphone) et ton rôle bascule vers : filmer/roder la démo terrain sur la PWA + la vidéo plan B.
- Ce guide reste la référence des **écrans, des règles métier et du contrat d'API** — que la démo terrain finale tourne en natif OU sur la PWA, la logique (règle 4 ha, contrôles d'intégrité, statuts verbatim, routes `/api/*`) est identique.
- L'app native reste au **plan v2 post-jury** pour ce que le navigateur ne fait pas (GPS en tâche de fond, détection de position simulée native, synchro offline lourde).

1. Vision : le rôle exact de l'app mobile

1.1 Ce que l'app mobile EST

- **Le terrain.** L'app mobile est l'outil de l'utilisateur au bord du champ (producteur, pisteuse ou agent de coopérative — jamais « délégué », sauf « délégué à la protection des données » qui est un terme légal). Elle démontre le **golden path complet en 6 étapes** devant le jury.
- **La capture GPS RÉELLE.** C'est SA valeur unique : là où le web **simule** la cartographie (waypoints posés par minuterie), le mobile écoute la **vraie géolocalisation** (`watchPosition` / `Location.watchPositionAsync`) : point central < 4 ha, **tour de champ ≥ 4 ha** (l'utilisateur marche le périmètre, le téléphone enregistre les waypoints). C'est la règle RDUE.
- **Le scan caméra réel.** Le web (pointeur fin) bascule volontairement en « saisie manuelle » ; le mobile porte le vrai scan : **QR d'abord**, repli **OCR Gemini Vision** (photo → API).

1.2 Ce que l'app mobile N'EST PAS

- Pas l'espace coopérative complet (dashboard, import de registre, vue exportateur) : c'est le **web**.
- Pas la vérification publique de certificat : c'est `/verifier-certificat` sur le web (le QR du PDF y mène).
- **Aucune fonctionnalité de crédit, score financier, plafond ou prêt.** L'étape 6 est « Valorisation » (primes de durabilité, acheteurs premium, partage du dossier de conformité). C'est une frontière produit stricte et non négociable. Si ton code actuel contient une étape crédit : **elle saute**.

1.3 Répartition web ↔ mobile (à connaître par cœur pour le jury)

	Web (agrivo-io.vercel.app)	Mobile (toi)
Golden path 6 étapes	Simulation propre (démonstration de secours)	Démonstration principale, GPS + caméra réels
Dashboard coop + import registre	✓	✗ (léger récap post-vérification au plus)
Vue exportateur + copilote IA	✓	✗
Vérification publique certificat	✓	✗ (le QR généré pointe vers le web)
Hors-connexion	PWA (cache)	Offline-first + file de synchronisation

2. Le golden path mobile, écran par écran

Règles transverses (tous les écrans) : statuts **verbatim** (« Conforme », « Anomalie détectée », « Données insuffisantes ») · toujours « évaluation », jamais « garantie » · aucun pourcentage de précision inventé · FR d'abord (EN si le temps le permet) · reduced-motion respecté (états finaux directs) · aucun appel Gemini/Whisp direct : **tout passe par les routes API AGRIVO** (§4).

Écran A — Connexion & sélection de la coopérative (étape 1)

- **Objectif** : entrer dans un contexte de travail : utilisateur + coopérative + consentement.
- **Comportement** : connexion simple (email/mot de passe, compte démo en 1 clic `client@test.com / 123client123` = Amadou, Coopérative Agricole de Soubré). Puis écran « Consentement enregistré » : rappel que le consentement éclairé du producteur est recueilli (ARTCI, loi n°2013-450), récap Coopérative / Gérant / Consentement, CTA « **Commencer le scan** ».
- **Logique métier** : la session est locale (pas de backend d'auth cette édition — le web fait pareil via localStorage). Coop de démo : `COOP-SOU` · Coopérative Agricole de Soubré.
- **Cas particuliers** : session expirée/absente → retour connexion. Mode démo TOUJOURS accessible (le jury ne doit jamais être bloqué par un mot de passe).
- **Offline** : la connexion démo fonctionne hors ligne (validation locale).
- **États** : chargement bref simulé (ressenti « vrai SaaS »), erreurs inline (email inconnu, mdp).

Écran B — Scan de la carte producteur (étape 2)

- **Objectif** : identifier le producteur en < 10 secondes, sans saisie.
- **Comportement utilisateur** : viseur caméra plein écran, cadre de visée (coins verts), bouton « **Scanner la carte** ». Pendant la lecture : ligne de balayage animée + « Lecture en cours... ». Résultat → **formulaire pré-rempli éditable** (Nom, N° de carte, Localité, Filière) + boutons « Confirmer » / « Rescanner ».
- **Logique métier (ordre EXACT)** :

1. **QR d'abord** : décodage local (ML Kit / expo-camera barcode ou Vision Camera code-scanner). Le QR des cartes de démo contient un JSON : `{"producteurNom": "...", "numeroCartePro": "CI-CCC-...", "localite": "...", "filiere": "cacao"}`. Si décodé → chip « Lues depuis le QR code de la carte », AUCUN appel réseau.
 2. **Repli OCR** : photo (JPEG ≤ 1280 px de large, base64 **sans** préfixe `data:`) → `POST /api/gemini/scan` → champs extraits. Couvre les cartes sans QR ou QR abîmé.
 3. **Anti-doublon matricule** : si le n° de carte correspond à un producteur connu → message « Producteur reconnu : dossier de <nom> rattaché, aucun doublon créé. » ; sinon « Nouveau matricule : unicité vérifiée dans le registre de la coopérative. »
 4. **La photo est conservée comme pièce justificative** (stockage local, rattachée au dossier).
- **Règles** : filière ∈ {cacao, cafe, hevea, palmier, bovins, soja, bois} (7 denrées RDUE) ; défaut `cacao` si illisible. Carte de démo pour la répétition : **Kouassi Yao · CI-CCC-024517 · Soubré**.
 - **Cas particuliers** : caméra refusée → mode démonstration (carte mock à l'écran) + saisie manuelle ; OCR live qui échoue → le serveur renvoie le résultat pré-enregistré (repli transparent).
 - **Offline** : QR = 100 % local, fonctionne offline. OCR indisponible → saisie manuelle proposée.
 - **Notifications/haptique** : vibration courte à la détection du QR (feedback terrain).
 - **Erreurs** : timeout API 12 s → message sobre + « Réessayer » + bascule saisie manuelle.

Écran C — Cartographie GPS de la parcelle (étape 3) — LE cœur mobile

- **Objectif** : produire la géométrie légale de la parcelle selon la règle RDUE : **point GPS (< 4 ha) ou polygone complet (≥ 4 ha)**, WGS-84, 6 décimales.
- **Trois modes** (cartes radio, recommandation automatique selon la superficie estimée) :
 1. **Point central** (recommandé < 4 ha) : l'utilisateur se place au centre du champ ; l'app moyenne les fixes GPS pendant quelques secondes (accuracy affichée ±m) et enregistre UN point.
 2. **Tour de champ GPS** (requis dès 4 ha) : l'utilisateur marche le périmètre ; `watchPositionAsync` ({accuracy: Highest, distanceInterval: ~8-10 m}) pose les **waypoints** ; compteurs **live** : waypoints posés · distance parcourue (haversine) · précision GPS ±m ; bouton « Fermer le polygone » actif après ≥ 3 sommets et retour près du départ (< ~15 m) — l'anneau est fermé automatiquement (premier point répété en dernier, GeoJSON).
 3. « **J'ai déjà les coordonnées** » (données existantes de la coop, ajouté en v1.1.0 web) : textarea « latitude, longitude » par ligne — 1 ligne = point ; ≥ 3 lignes = polygone fermé auto. Validation : format numérique, **emprise Côte d'Ivoire (lat 4–11, lon -9 à -2)** — messages d'erreur exacts du web : « Format invalide... », « Ces coordonnées sortent de l'emprise de la Côte d'Ivoire (lat 4–11, lon -9–-2). Vérifiez l'ordre latitude, longitude. »
- **Carte** : fond satellite (Esri World Imagery), tracé du polygone en cours (polyline verte), position live, waypoints. Le web utilise Leaflet + tuiles Esri ; en RN : `react-native-maps` (mapType satellite) ou MapLibre + raster Esri.
- **Contrôles d'intégrité (anti-fraude, à afficher en 4 coches séquentielles après capture)** :
 1. « Aucun chevauchement avec les parcelles déjà enrôlées » (intersection d'emprises, local) ;
 2. « Superficie plausible : plafond d'achat X t/an (filière, rendement régional) » — X = superficie × rendement {cacao 0,6 · cafe 0,5 · hevea 1,4 · palmier 3,0 t/ha} ;
 3. « Signal GPS authentique : aucune position simulée détectée » — **sur mobile c'est RÉEL** : Android `isFromMockProvider / isMock` (expo-location `mocked`) ; iOS : plausibilité (vitesse/accuracy). Si mock détecté → blocage avec message honnête ;
 4. « Matricule <n°> : unique, aucun doublon ».
- **Règles** : superficie calculée depuis le polygone (formule de l'aire géodésique) et comparée à la déclaration ; précision cible ≤ ±10 m par waypoint (sinon avertir) ; anneau fermé obligatoire.

- **Cas particuliers** : perte de signal en plein tour → pause + reprise (les waypoints restent) ; précision > 25 m persistante → proposer de refaire ; parcelle < 4 ha mais l'utilisateur veut un polygone → autorisé (le point est le minimum légal, pas le maximum).
- **Offline** : la capture GPS est 100 % locale → TOUJOURS disponible offline. La géométrie est mise en **file de synchronisation** pour l'analyse (écran D) dès le retour réseau.
- **États** : « Tour de champ en cours » avec compteurs live ; « Acquisition du point central » ; puis récap « Polygone fermé : N sommets, coordonnées WGS-84 (GeoJSON RFC 7946) · Superficie : X ha ».

Écran D — Analyse satellite Whisp (étape 4)

- **Objectif** : le verdict. LE moment signature de la démo.
- **Comportement** : bouton « Lancer l'analyse » → séquence : contour de la parcelle qui se dessine sur l'imagerie satellite (~0,6 s) → balayage « Analyse satellite en cours... » (~0,8 s + latence API) → **verdict** : badge statut + phrase figée + **faisceau de preuves** (3 puces qualitatives renvoyées par l'API) + bouton « Écouter » (TTS natif, fr-FR) + badge « Score de résilience des sols » (popover explicatif via `/api/gemini/explain`).
- **Logique métier** : `POST /api/whisp/verify` avec `parcelleId` (parcours démo) OU `coordinates` (géométrie capturée). Les trois statuts et leurs phrases sont FIGÉS (renvoyés par l'API — ne jamais les reformuler côté client) :
-  **Conforme** — « Aucune déforestation détectée après le 31 décembre 2020. »
-  **Anomalie détectée** — « Une perte de couverture forestière a été identifiée sur cette zone. »
-  **Données insuffisantes** — « Présence de nuages ou données satellites insuffisantes pour statuer. »
- **Branchements** (identiques au web) : `insuffisant` → écran de fin direct (PAS de certificat) ; `anomalie` → certificat documentaire puis fin (PAS de valorisation) ; `conforme` → certificat puis Valorisation.
- **Démo & maîtrise du verdict** : le body accepte `force: "conforme" | "anomalie" | "insuffisant"` (guide présentateur) — utile en répétition. En mode démo serveur, la latence est simulée (1,2–1,8 s).
- **Offline** : si pas de réseau → la vérification part en **file d'attente** avec état « En attente de synchronisation » ; à la reconnexion, POST automatique + notification locale « Verdict disponible ».
- **Erreurs** : timeout/HTTP ≥ 500 → « L'analyse n'a pas abouti. Réessayer. » (pas de verdict inventé).

Écran E — Certificat (étape 5)

- **Objectif** : la preuve tangible : certificat d'évaluation de conformité RDUE.
- **Comportement** : animation de génération (~1 s), puis **aperçu du certificat** : en-tête Agrivo, n° de certificat (`AGV-2026-XXXX`), date/heure, statut teinté (vert/rouge/ambre) + phrase figée, producteur, n° de carte, coopérative, filière, région, superficie, **coordonnées WGS-84 (6 décimales, RFC 7946)** listées sommet par sommet, sources de données, mention TRACES NT + consentement ARTCI, avertissement légal (« Évaluation technique... ne se substitue pas à la responsabilité légale de l'opérateur »). Boutons « Télécharger le PDF » / « Continuer ».
- **Logique** : le PDF contient un **QR code** qui pointe vers `https://agrivo-io.vercel.app/verifier-certificat?ref=<n°>` — c'est l'« effet final » du jury (ils scannent avec LEUR téléphone). Sur mobile : générer le PDF localement (lib PDF RN) OU partager le lien de vérification + aperçu natif ; l'important pour la démo est le **QR affiché à l'écran**.
- **Cas particuliers** : échec de génération PDF → l'aperçu reste, le partage du lien suffit.
- **Offline** : l'aperçu et le QR se génèrent localement (les données sont déjà là).

Écran F — Valorisation (étape 6, uniquement si Conforme)

- **Objectif** : transformer la conformité en argument commercial. AUCUN crédit.
- **Comportement** : « La parcelle de <nom> rejoint le dossier de valorisation » + rappel parcelle (badge statut, n° certificat, superficie) + compteur « Portefeuille de la coopérative : X parcelles conformes sur Y vérifiées » + les 3 débouchés :

1. **Prime de durabilité négociable** (au-dessus du prix garanti — jamais « négocier le prix ») ;
 2. **Accès aux acheteurs premium** (certificat vérifiable par QR) ;
 3. **Dossier prêt pour TRACES NT** (géolocalisation, sources satellite, diligence). CTA « **Partager le dossier avec l'exportateur** » → simulation d'envoi (~1,4 s) → écran de succès (pin + coche, « Dossier partagé ») → « Retour au tableau de bord ».
- **Interdits** : aucun montant, aucun slider, aucun score financier, aucun plafond. Si l'existant a l'ancien écran « Crédit » (slider 50 000–250 000 FCFA, Mobile Money) : **le supprimer entièrement**.
 - **Offline** : le partage part en file de synchro comme le reste.

Écran G — Fin de parcours

- Récap adaptatif selon verdict (messages exacts du web) : conforme → « La parcelle est conforme, le certificat est prêt pour TRACES NT. » ; anomalie → « L'anomalie a été signalée... » ; insuffisant → « L'analyse n'est pas concluante : un nouveau passage satellite est nécessaire. Aucun certificat n'est émis à ce stade. » + boutons « Retour au tableau de bord » / « Nouvelle vérification ».

3. Architecture mobile recommandée

3.1 Stack (recommandation ferme, sauf si ton code de départ impose autre chose de solide)

- **React Native + Expo (SDK récent) + TypeScript strict** — cohérence maximale avec le web (React 19, mêmes types partageables, même vocabulaire), OTA updates (EAS Update) pratiques à 5 jours du jury, et accès natif suffisant (caméra, GPS, haptique) sans écrire de natif.
- Si ton existant est **Flutter** ou bare RN : garde-le SEULEMENT s'il fait déjà tourner caméra + GPS + carte satellite ; sinon la migration Expo est plus rapide que le rattrapage.
- **Navigation** : `expo-router` (file-based, cohérent avec App Router web) ou React Navigation (stack). Le parcours est un **stack modal linéaire** avec état partagé, pas des écrans isolés.
- **État** : machine à états du parcours (copie du web : `step 1→7`, branchements par verdict) dans un store **zustand** (léger) ; réseau via **TanStack Query** (retry, cache, states).
- **Cartes** : `react-native-maps` (mapType: satellite/hybrid) OU MapLibre GL + raster `https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}` (même imagerie Esri que le web ; attribution Esri requise à l'écran).
- **Caméra + QR** : `expo-camera` (scan QR intégré) ; photo → base64 JPEG ≤ 1280 px.
- **GPS** : `expo-location` — `watchPositionAsync({ accuracy: Highest, distanceInterval: 8 })`, champ `mocked` pour le verrou anti-simulation (Android).
- **Stockage local / file de synchro** : MMKV (clé-valeur rapide) + une table SQLite (`expo-sqlite`) `sync_queue(id, type, payload, createdAt, status)` — voir §3.4.
- **Animations** : Reanimated 3 + Moti, sobres (150–250 ms), reduced-motion respecté (`AccessibilityInfo.isReduceMotionEnabled`).

3.2 Structure de dossiers (miroir du web pour s'y retrouver)

```
app/                # expo-router : (auth)/connexion, (verifier)/step-[1..6]
components/verifier/ # step-scan, step-mapping, step-analysis, step-certificate, step-valorisation
components/ui/       # StatusBadge, PinMark, boutons (mêmes noms que le web)
lib/api.ts           # client HTTP typé (fetch + timeout 12 s + retry 1)
lib/types.ts         # ScanResult, WhispResult, Statut... (copiés du web, source de vérité)
lib/geo.ts           # haversine, aire géodésique, fermeture d'anneau, emprise CI
lib/sync-queue.ts    # file offline (enqueue, flush on reconnect)
config/brand.ts      # BRAND_NAME, couleurs (tokens ci-dessous)
data/demo.ts         # parcelle démo Kouassi Yao CI-CCC-024517, coop COOP-SOU, Soubré [-6.65, 5.83]
```

3.3 Design tokens (les MÊMES que le web — aucune invention)

- Fond sombre #0a1f14 (forest-950) · vert signal #16a34a (Conforme) · rouge #b4231e (Anomalie détectée) · ambre #c8861d / #e0a64b (Données insuffisantes / accents) · ivoire #f7f3ea · encre #142019.
- Titres **Space Grotesk** (droit, jamais italique, max 700) · corps système ou Geist · chiffres et références en **monospace**.
- Statuts TOUJOURS icône + texte (jamais la couleur seule).

3.4 File de synchronisation offline (spécification)

- Toute action réseau (analyse Whisp, partage dossier, mémo) est **enqueue** si offline : `{type: 'whisp' | 'partage', payload, status: 'pending'}`.
- Écouteur réseau (NetInfo) → au retour : flush FIFO, 1 retry avec backoff, puis marquage `failed` avec relance manuelle. Notification locale au succès (« Verdict disponible : Conforme »).
- L'UI montre TOUJOURS l'état : chip « En attente de synchronisation » sur les dossiers en file.
- Les photos de cartes et géométries restent locales tant que non synchronisées (rien ne se perd).

3.5 Permissions & confidentialité

- Permissions demandées **au moment de l'usage** (caméra à l'écran Scan, localisation à l'écran Cartographie), avec phrase d'usage claire (ARTCI) : « Votre position sert uniquement à cartographier la parcelle. Vos données restent la propriété de la coopérative. »
- Localisation : `WhenInUse` suffit (pas de background). Refus → mode « J'ai déjà les coordonnées » reste utilisable (l'app ne bloque jamais).

3.6 Sécurité de la clé Gemini (RÈGLE ABSOLUE)

- La clé `GEMINI_API_KEY` ne doit **JAMAIS** exister dans l'app mobile (ni en dur, ni en env bundlée, ni en fichier de config — un APK se décompile en 2 minutes).
- L'app appelle **exclusivement** les routes AGRIVO (`/api/gemini/*` , `/api/whisp/verify`) : la clé vit côté serveur (Vercel). C'est déjà l'architecture du web — le mobile la réutilise telle quelle.
- Pas de secret dans le code : la seule config est `API_BASE = "https://agrivo-io.vercel.app"`.

4. Contrat d'API (backend existant, vérifié dans le code v1.1.0)

Base : `https://agrivo-io.vercel.app` · Tous les endpoints : `POST`, `Content-Type: application/json`. Mode serveur : **LIVE** si la clé Gemini est posée (c'est le cas en prod), sinon **MOCK** automatique. Chaque route garde un repli pré-enregistré si l'appel live échoue (sauf memo : 503, voir plus bas). Timeout côté serveur : 12 s. Latence simulée en mock : ~1,2–1,8 s (ressenti réel). ⚠ Aucune authentification d'API cette édition (démon). Ne PAS y envoyer de données personnelles réelles hors démon/répétition.

4.1 `POST /api/whisp/verify` — verdict satellite

Requête (tous champs optionnels, mais envoyer `parcelleId` OU `coordinates`) :

```
{
  "parcelleId": "p01",
  "coordinates": [[[ -6.647800, 5.832100 ], [ -6.646100, 5.833000 ], [ -6.645200, 5.831800 ], [ -6.647800, 5.832100 ]]],
  "filier": "cacao",
  "force": "conforme"
}
```

- `coordinates` : GeoJSON **RFC 7946** — ordre [**longitude, latitude**], **6 décimales**, anneau **fermé** (premier point répété en dernier). Un point simple : [`lon`, `lat`].
- `force` (démon/présentateur) : impose le statut.

Réponse `200` :

```
{
  "statut": "conforme",
  "phrase": "Aucune déforestation détectée après le 31 décembre 2020.",
  "datePivot": "2020-12-31",
  "sources": ["Copernicus Sentinel-2", "Whisp · FAO Open Foris", "JRC Global Forest Cover"],
  "convergence": ["Imagerie optique Sentinel-2 (Copernicus) : couvert végétal stable depuis la date pivot.", "...", "..."],
  "analyseMs": 1450,
  "demo": true
}
```

- `statut` ∈ `conforme` | `anomalie` | `insuffisant`. Afficher `phrase` telle quelle (jamais reformulée).
- **Depuis v1.2.0**, la réponse contient AUSSI `phraseEn` (phrase figée anglaise) et `convergenceEn` (les 3 preuves en anglais) : si tu fais une interface EN, choisis le champ selon la langue — ne traduis JAMAIS toi-même. Pour le TTS : `fr-FR` avec `phrase`, `en-GB` avec `phraseEn`.

4.2 `POST /api/gemini/scan` — OCR de la carte producteur

Requête : `{ "imageBase64": "<JPEG base64 SANS préfixe data:>", "mimeType": "image/jpeg" }` (body vide accepté → résultat pré-enregistré). Réponse `200` : `{ "producteurNom": "Kouassi Yao", "numeroCartePro": "CI-CCC-024517", "localite": "Soubré, région du Nawa", "filier": "cacao", "live": true }`

- `live` présent seulement si l'OCR Gemini a réellement tourné. `filiere` est déjà validée serveur (repli `cacao`).
Champs illisibles → chaînes vides : garder le formulaire éditable.

4.3 POST /api/gemini/memo — dossier de diligence (DDS)

Requête : `{ "parcelleId": "p01" }` Réponses : `200` = `{ reference, producteur, statut, genereLe, sections: [{titre, corps}], conclusion, avertissement, live? }` · `404` = parcelle inconnue · `503 {"error":"retry"}` = **IA live indisponible** → afficher « L'IA est momentanément indisponible. Veuillez réessayer plus tard. » (décision v1.1.0 : pas de repli silencieux en live). Usage mobile : optionnel (écran parcelle) — pas nécessaire à la démo.

4.4 POST /api/gemini/query — copilote portefeuille (web exportateur ; mobile : non requis)

Requête : `{ "question": "Quelles parcelles présentent un risque dans la région de Soubré ?" }` Réponse : `{ texte, parcelles: [...], metric?: {label, value}, tools: [{name, detail}], analyseMs, live? }`

4.5 POST /api/gemini/explain — explications (verdict / score sols)

Requête : `{ "kind": "sols", "producteurNom": "Kouassi Yao" }` → `{ "niveau": "Élevé" | "Moyen" | "À renforcer", "explication": "..." }` ou `{ "kind": "verdict", "statut": "conforme" }` → `{ "explication": "..." }` (phrase figée). Formulation imposée du score sols : « méthodologie inspirée de standards reconnus type Kubeko » (jamais « partenaire Lono »).

4.6 Quoi de neuf backend v1.2.0/v1.2.1 (7 juillet) — ce qui te concerne

Deux nouvelles routes IA existent en prod (vérifiées live). **Aucune des deux n'est requise pour ta démo mobile** — elles vivent sur le web (dashboard + étape 6). À connaître pour le jury, et utilisables si tu as fini en avance :

- POST /api/gemini/valorisation-memo — **optionnel écran F (Valorisation)** : body `{ "parcelleId": "p01", "lang": "fr" }` → `{ titre, paragraphes: [4 × string], live? }`. Gemini rédige l'argumentaire de prime de la coopérative (aucun montant promis, zéro vocabulaire de crédit). Si `live` est absent/false : afficher un badge « Mode démonstration » (honnêteté).
- POST /api/gemini/audit-plan — web uniquement (plan d'action après l'audit du registre, qui n'existe pas sur mobile) : body `{ resume: {...}, lang }` → `{ etapes, conclusion, live? }`.
- GET /api/admin/etat → `{ "mock": false, "model": "gemini-2.5-flash" }` — pratique pour vérifier depuis ton téléphone que le backend est en mode IA live avant une répétition.
- ⚠ La page **/connexion n'affiche plus les identifiants démo** (durcissement v1.2.1) : ils sont dans ce document (§4.7) et le bouton 1-clic reste. Le discours jury passe à « **5 usages IA en production** » (OCR, plan d'action d'audit, mémo DDS, argumentaire de prime, copilote).

4.7 Données de démo à connaître

- Parcelle fil rouge : `p01` = **Kouassi Yao · CI-CCC-024517 · cacao · Soubré** (conforme).
- Zone de démo : Soubré rural `[lon -6.65, lat 5.83]` (le serveur court-circuite le réseau dans un rayon large autour de `[-6.6039, 5.7853]`).
- Comptes : `client@test.com / 123client123` (Amadou) · `admin@agrivo.com / 123admin123`.
- Emprise CI acceptée par la validation : lat 4–11, lon -9 à -2.

5. UX mobile (exigences)

- **Navigation** : stepper 6 étapes toujours visible (compact, scrollable), retour possible à chaque étape (« Retour » texte, sans détruire l'état), jamais de swipe destructif.
- **Terrain d'abord** : boutons ≥ 48 px, actions principales en bas d'écran (pouce), contrastes forts (soleil), textes courts, une action principale par écran.
- **Feedback permanent** : chaque attente a un état nommé (« Lecture en cours... », « Tour de champ en cours », « Analyse satellite en cours... », « Partage du dossier... ») ; haptique légère aux jalons (QR lu, polygone fermé, verdict) ; compteurs live pendant le tour de champ (waypoints, distance, $\pm m$).
- **Animations** : sobres et signifiantes (150–250 ms) ; le budget créatif se concentre sur UNE séquence : **l'analyse satellite** (contour → balayage → verdict). Reduced-motion : états finaux.
- **Accessibilité** : statuts icône + texte, labels de boutons explicites, TTS du verdict (fr-FR), tailles dynamiques respectées, focus/ordre de lecture cohérents.
- **Offline visible** : bandeau discret « Hors connexion — vos captures sont enregistrées localement » + chips « En attente de synchronisation ».
- **Langue** : FR par défaut. EN seulement si tout le FR est parfait avant jeudi soir.

6. Ce que le jury doit voir (critères de démo, vendredi 10 en répétition)

1. Scan QR d'une carte → formulaire pré-rempli en < 10 s (chip « Lues depuis le QR code »).
2. Tour de champ GPS RÉEL dans une cour/parking : waypoints qui se posent en marchant, distance et précision live, polygone fermé → 4 contrôles d'intégrité cochés.
3. Verdict Whisp avec badge + phrase figée + preuves (latence réaliste, jamais d'attente > 4 s).
4. Certificat avec QR à l'écran → le jury scanne avec SON téléphone → `/verifier-certificat` web.
5. Valorisation : « Partager le dossier avec l'exportateur » → succès.
6. Mode avion pendant l'étape C → tout continue, la vérification se synchronise au retour du réseau.

7. Réadapter TON existant (plan de travail)

7.1 Questions auxquelles répondre (envoie-moi les réponses, 10 min)

1. Stack exacte : Expo ou bare RN ? Version SDK ? (ou Flutter ?) TypeScript ?
2. Navigation actuelle : quelles routes/écrans existent déjà ?
3. Le scan : caméra branchée ? QR décodé localement ? OCR appelé où (direct Gemini ⚠ ou via notre API) ?
4. La carto : `watchPosition` déjà utilisé ? Y a-t-il déjà une carte satellite ? Quelle lib ?
5. L'analyse : appelles-tu `https://agrivo-io.vercel.app/api/whisp/verify` ou un mock local ?
6. Reste-t-il un écran **Crédit** (slider FCFA, Mobile Money) ? → à supprimer.
7. Des textes « garantie », des pourcentages de précision, « délégué » ? → à purger.
8. Une clé API quelconque dans le code/env de l'app ? → à retirer immédiatement.
9. Offline : quelque chose d'existant (cache, file) ?
10. Sur quel téléphone fais-tu la démo ? (Android recommandé : BarcodeDetector/mock-location OK.)

7.2 Plan d'alignement progressif (ordre imposé, 3 jours)

- **Lot 1 (mardi) — Conformité charte & API** : supprimer tout crédit ; brancher TOUTES les IA sur les routes AGRIVO (aucune clé locale) ; statuts verbatim + phrases figées ; écrans A/B fonctionnels.

- **Lot 2 (mercredi) — Le cœur : cartographie réelle** : watchPosition, waypoints, compteurs live, fermeture d'anneau, contrôles d'intégrité, mode « coordonnées existantes ». Test terrain réel.
- **Lot 3 (jeudi matin) — Verdict + certificat + valorisation** : séquence d'analyse, QR de vérification publique, écran Valorisation, écran de fin. File offline minimale (whisp).
- **Lot 4 (jeudi soir) — Polish démo** : latences, haptique, états d'erreur, répétition du déroulé, **tournage de la vidéo plan B** (plans 2, 3, 5 du script — voir GUIDE_DEMO_JURY.md).
- **Gel vendredi midi**. Répétition générale vendredi (P9) avec le web.

7.3 Critères d'acceptation par écran (cocher AVANT la répétition)

Écran	Critères (tous requis)
A Connexion	☐ compte démo 1 clic · ☐ récap consentement ARTCI · ☐ offline OK
B Scan	☐ QR < 10 s · ☐ repli OCR via <code>/api/gemini/scan</code> · ☐ formulaire éditable · ☐ anti-doublon · ☐ photo conservée · ☐ refus caméra géré
C Carto	☐ point central · ☐ tour de champ waypoints live · ☐ coordonnées collées + validation CI · ☐ 4 contrôles d'intégrité · ☐ anneau fermé 6 décimales · ☐ mock-location bloqué · ☐ offline OK
D Analyse	☐ POST whisp avec vraie géométrie · ☐ 3 statuts + phrases verbatim · ☐ branchements corrects · ☐ erreur réseau propre · ☐ file offline
E Certificat	☐ toutes les mentions (n°, coords 6 déc., sources, TRACES NT, avertissement) · ☐ QR scannable vers <code>/verifier-certificat</code>
F Valorisation	☐ zéro vocabulaire financier · ☐ 3 débouchés · ☐ partage simulé + succès
G Fin	☐ 3 variantes de message selon verdict

7.4 Gates avant de dire « c'est prêt »

- `tsc --noEmit` sans erreur (si TS) · lint propre · l'app démarre à froid en < 3 s sur ton téléphone.
- Déroulé complet des 6 étapes SANS toucher au code, 3 fois de suite, dont 1 fois en mode avion.
- Grep du code : `crédit|credit|prêt|loan|FCFA|garantie|guarantee|délégué` → zéro occurrence UI (hors « délégué à la protection des données » si l'écran légal existe).

AGRIVO — document technique interne · 6 juillet 2026, mis à jour le 7 juillet (backend v1.2.1) · rédigé par Anael pour Christ. Jumeau : `AGRIVO_Guide_App_Mobile.pdf`. Références : `CLAUDE.md` , `PLAN_REORIENTATION_AGRIVO.md` , `GUIDE_DEMO_JURY.md` , code web `components/verifier/` et `app/api/` (source de vérité du contrat).